

فاز اول

کیوان ثاقب مجیدزاده

موضوع برنامه

هدف این برنامه دریافت یک فرمول منطقی گزاره‌ای و تولید سه نمایش متفاوت از آن است:

۱. فرم نرمال عطفی (CNF)؛

۲. فرم نرمال فصلی (DNF)؛

۳. فرم ساده‌شده معادل.

برنامه ابتدا متن ورودی را تجزیه می‌کند و برای آن یک syntax tree می‌سازد. سپس عملگرهای شرطی و دوشروطی را حذف می‌کند، فرمول را به NNF می‌برد و با توزیع عملگرها، دو خروجی CNF و DNF را تولید می‌کند. در پایان نیز قوانین ساده‌سازی روی فرمول اعمال می‌شوند.

ورودی و خروجی

ورودی برنامه تنها یک خط و شامل یک فرمول منطقی است. برای نمونه:

$(\neg p \wedge \neg q)$

برنامه سه خط خروجی چاپ می‌کند. ترتیب این سه خط به صورت زیر است:

۱. CNF؛

۲. DNF؛

۳. فرم ساده‌شده معادل.

برای ورودی بالا، خروجی برنامه چنین است:

$(p \vee q)$

$(p \vee q)$

$(p \vee q)$

نمادهای قابل قبول

نماد در ورودی	معنی
a تا z	متغیر گزاره‌ای تک حرفی
T	ثابت درست
F	ثابت نادرست
$\neg A$	نقیض A
$(A \& B)$	عطف $A \wedge B$
$(A B)$	فصل $A \vee B$
$(A \rightarrow B)$	شرطی $A \rightarrow B$
$(A \leftrightarrow B)$	دوشرطی $A \leftrightarrow B$

هر زیرفرمول دوتایی باید داخل یک جفت پرانتز نوشته شود. برای مثال، $(p \& q)$ معتبر است، اما $p \& q$ با دستور زبان فعلی برنامه پذیرفته نمی‌شود. همچنین برنامه فاصله‌های خالی داخل فرمول را حذف نمی‌کند؛ بنابراین ورودی باید بدون فاصله نوشته شود.

ساختار کلی اجرای برنامه

مراحل اجرای برنامه به ترتیب زیر است:

۱. خواندن یک خط از ورودی در تابع `main`؛
۲. ساخت `syntax tree` به کمک کلاس `Parser`؛
۳. حذف عملگرهای شرطی و دوشرطی؛
۴. تبدیل نتیجه به `NNF`؛
۵. تولید `CNF` با تابع `make_cnf`؛
۶. تولید `DNF` با تابع `make_dnf`؛
۷. ساده سازی فرمول تا رسیدن به نقطه ثابت؛
۸. تبدیل ساختارهای داخلی به متن و چاپ سه نتیجه.

تابع `solve` هماهنگ کننده اصلی این مراحل است:

```
root = Parser(text).parse()
without_implications = eliminate_implications(root)
nnf = to_nnf(without_implications)
cnf = make_cnf(nnf)
dnf = make_dnf(nnf)
simplified = simplify_to_fixed_point(nnf)
```

نمایش فرمول با `syntax tree`

کلاس داده ای `Node` هر گره از `syntax tree` فرمول را نمایش می دهد. این کلاس چهار ویژگی دارد:

ویژگی	توضیح
kind	نوع گره، مانند <code>var</code> ، <code>const</code> ، <code>not</code> ، <code>and</code> ، <code>or</code> ، <code>imp</code> یا <code>iff</code>
value	مقدار متغیر یا ثابت؛ برای مثال <code>p</code> ، <code>T</code> یا <code>F</code>
left	فرزند چپ در عملگرهای دوتایی یا تنها فرزند در گره نقیض
right	فرزند راست در عملگرهای دوتایی

استفاده از گزینه `frozen=True` باعث می شود اشیای `Node` پس از ساخته شدن تغییر نکنند. این ویژگی دو مزیت دارد: نخست، تبدیل های مختلف به جای تغییر درخت قبلی، درخت جدیدی می سازند؛ دوم، گره ها قابل ذخیره در `set` هستند و می توان از آن برای حذف موارد تکراری استفاده کرد.

توابع `make_var`، `make_const`، `make_not` و `make_binary` سازنده های کمکی هستند و ایجاد انواع مختلف گره را خواناتر می کنند.

تجزیه فرمول

کلاس Parser متن فرمول را از چپ به راست بررسی می کند. متغیر pos محل فعلی خواندن متن را نگه می دارد. متد parse_formula به صورت بازگشتی یکی از حالت های زیر را تشخیص می دهد:

$$F ::= a \mid b \mid \dots \mid z \mid T \mid F \\ \mid !F \\ \mid (F \& F) \mid (F \mid F) \mid (F \rightarrow F) \mid (F \leftrightarrow F).$$

اگر نویسه فعلی یک حرف کوچک باشد، یک گره متغیر ساخته می شود. اگر T یا F باشد، یک گره ثابت ساخته می شود. مشاهده عملگر نقیض ! باعث فراخوانی بازگشتی تابع برای زیرفرمول بعدی می شود. در صورت مشاهده پرانتز باز، برنامه ابتدا فرمول سمت چپ، سپس عملگر و بعد فرمول سمت راست را می خواند.

عملگرهای بلندتر پیش از عملگرهای کوتاه تر بررسی می شوند. به همین دلیل <-> پیش از >- آزمایش می شود تا بخشی از یک عملگر بلند به اشتباه به عنوان عملگر کوتاه تر تشخیص داده نشود.

متد parse پس از ساخت syntax tree بررسی می کند که همه نویسه های ورودی مصرف شده باشند. در صورت وجود ورودی اضافی، عملگر نامعتبر یا پرانتز بسته ناقص، خطای ValueError ایجاد می شود.

حذف عملگرهای شرطی و دوشروطی

تابع `eliminate_implications`، syntax tree را به صورت بازگشتی پیمایش می کند. عملگرهای عطف، فصل و نقیض بدون تغییر معنایی بازسازی می شوند، اما عملگرهای شرطی و دوشروطی با رابطه های معادل زیر جایگزین می شوند:

$$A \rightarrow B \equiv \neg A \vee B$$

$$A \leftrightarrow B \equiv (\neg A \vee B) \wedge (\neg B \vee A).$$

پس از اجرای این مرحله، syntax tree فقط شامل متغیرها، ثابت ها، نقیض، عطف و فصل است. این محدودسازی، تبدیل مرحله بعد به NNF را ساده می کند.

تبدیل به NNF

در NNF، نقیض فقط مستقیماً پیش از متغیرها و ثابت ها قرار می گیرد. تابع `to_nnf` برای انتقال نقیض به سطوح پایین تر syntax tree از قوانین دمورگان استفاده می کند:

$$\neg(A \wedge B) \equiv \neg A \vee \neg B,$$

$$\neg(A \vee B) \equiv \neg A \wedge \neg B.$$

همچنین نقیض مضاعف و نقیض ثابت ها به صورت زیر پردازش می شوند:

$$\neg\neg A \equiv A, \quad \neg T \equiv F, \quad \neg F \equiv T.$$

پارامتر `is_negated` مشخص می کند که گره فعلی زیر اثر یک نقیض قرار دارد یا خیر. هنگام عبور از یک گره `not`، مقدار این پارامتر معکوس می شود. این روش بدون ساخت زنجیره های اضافی از نقیض، فرمول را در یک پیمایش بازگشتی به NNF می برد.

ساخت فرم نرمال عطفی

CNF عطفی از چند clause است که هر clause، فصلی از چند literal است:

$$(L_{11} \vee L_{12} \vee \dots) \wedge (L_{21} \vee L_{22} \vee \dots) \wedge \dots$$

برنامه CNF را به شکل یک فهرست از فهرست ها ذخیره می کند. فهرست بیرونی شامل clause ها است و هر فهرست داخلی، literal های یک clause را نمایش می دهد. برای نمونه، فرمول

$$(p \vee q) \wedge (\neg p \vee r)$$

به صورت مفهومی چنین ذخیره می شود:

```
[
  [p, q],
  [!p, r]
]
```

در تابع `make_cnf` سه حالت وجود دارد:

- یک literal به یک clause تک‌عضوی تبدیل می‌شود؛
- برای $A \wedge B$ ، clause‌های دو طرف به یکدیگر متصل می‌شوند؛
- برای $A \vee B$ ، ضرب دکارتی clause‌های دو طرف محاسبه می‌شود.

حالت سوم همان توزیع عملگرها است. اگر $A = C_1 \wedge C_2$ و $B = D_1 \wedge D_2$ باشند، آنگاه:

$$A \vee B \equiv (C_1 \vee D_1) \wedge (C_1 \vee D_2) \wedge (C_2 \vee D_1) \wedge (C_2 \vee D_2).$$

ساخت فرم نرمال فصلی

DNF فصلی از چند term است که هر term، عطفی از چند literal است:

$$(L_{11} \wedge L_{12} \wedge \dots) \vee (L_{21} \wedge L_{22} \wedge \dots) \vee \dots.$$

در تابع `make_dnf` نقش عطف و فصل نسبت به تبدیل CNF جابه‌جا می‌شود:

- یک literal به یک term تک‌عضوی تبدیل می‌شود؛
- برای $A \vee B$ ، term‌های دو طرف به یکدیگر متصل می‌شوند؛
- برای $A \wedge B$ ، ضرب دکارتی term‌های دو طرف محاسبه می‌شود.

برای مثال، توزیع مورد استفاده در این بخش چنین است:

$$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C).$$

حذف موارد تکراری

تابع `deduplicate_nodes`، literal‌های تکراری داخل یک clause یا term را حذف می‌کند. تابع `deduplicate_groups` نیز clause‌ها یا term‌های کاملاً تکراری را حذف می‌کند. هر دو تابع با استفاده از set عمل می‌کنند، اما نتیجه را در یک فهرست جداگانه نگه می‌دارند تا ترتیب چپ‌به‌راست عناصر حفظ شود.

این مرحله فقط تکرارهای دقیق را حذف می‌کند. برای مثال، دو گروه یکسان p یکی می‌شوند؛ ولی قوانین پیشرفته‌تری مانند قانون جذب $p \vee (p \wedge q) \equiv p$ در این قسمت اجرا نمی‌شوند.

ساده سازی فرمول

تابع `simplify_once` ابتدا فرزندان هر گره را ساده می کند و سپس قوانین مناسب آن گره را اعمال می کند. مهم ترین قوانین ساده سازی استفاده شده شامل موارد زیر هستند:

قانون	نتیجه
$\neg T$	F
$\neg F$	T
$\neg \neg A$	A
$A \wedge F$	F
$A \wedge T$	A
$A \wedge A$	A
$A \wedge \neg A$	F
$A \vee T$	T
$A \vee F$	A
$A \vee A$	A
$A \vee \neg A$	T

تابع `is_negation_pair` بررسی می کند که آیا دو طرف یک عملگر، نقیض یکدیگر هستند یا خیر. این بررسی برای تشخیص `contradiction` یعنی $A \wedge \neg A$ و `tautology` یعنی $A \vee \neg A$ استفاده می شود.

ممکن است اجرای یک قانون، فرصت اجرای قانون دیگری را ایجاد کند. به همین دلیل تابع `simplify_to_fixed_point` ساده سازی را چند بار تکرار می کند تا خروجی یک مرحله با ورودی همان مرحله یکسان شود. در این وضعیت فرمول به نقطه ثابت رسیده است و قوانین موجود دیگر تغییری در آن ایجاد نمی کنند.

تبدیل نتیجه به متن

توابع زیر نمایش های داخلی را دوباره به رشته تبدیل می کنند:

- `render_literal`: تبدیل یک متغیر، ثابت یا نقیض یک literal؛
- `render_cnf`: ساخت متن clause های CNF و اتصال آن ها با &؛
- `render_dnf`: ساخت متن term های DNF و اتصال آن ها با |؛
- `render_formula`: تبدیل یک syntax tree عمومی به متن؛
- `fold_left_text`: پرانتزگذاری چپ چین برای چند عبارت متوالی.

برای مثال، تابع `fold_left_text` فهرست `p, q, r` را با عملگر & به شکل زیر تبدیل می کند:

$$((p \wedge q) \wedge r).$$

تابع `render_formula`، عملگرهای عطف یا فصل متوالی و یکسان را ابتدا در یک فهرست جمع می کند و سپس آنها را به صورت چپ چین چاپ می کند. این کار باعث یکنواخت شدن قالب خروجی می شود.

شرح تابع اصلی

تابع main یک خط را از ورودی استاندارد می‌خواند:

```
formula = sys.stdin.readline().strip()
```

متد strip نویسه پایان خط و فاصله‌های ابتدا و انتهای ورودی را حذف می‌کند. سپس تابع solve فراخوانی می‌شود و سه رشته خروجی را برمی‌گرداند:

```
cnf_text, dnf_text, simplified_text = solve(formula)
```

در پایان، هر نتیجه در یک خط جداگانه چاپ می‌شود. شرط `if __name__ == "__main__"` نیز تضمین می‌کند که main فقط هنگام اجرای مستقیم فایل فراخوانی شود و هنگام `import` شدن فایل به عنوان یک ماژول اجرا نشود.

اجرای نمونه اول

ورودی:

$(p \rightarrow q)$

در مرحله حذف عملگر شرطی داریم:

$$p \rightarrow q \equiv \neg p \vee q.$$

این فرمول از قبل هم در CNF و هم در DNF قرار دارد؛ بنابراین خروجی به شکل زیر است:

$(\neg p | q)$
 $(\neg p | q)$
 $(\neg p | q)$

اجرای نمونه دوم

ورودی:

$((p \& T) | F)$

قوانین ساده‌سازی زیر اعمال می‌شوند:

$$p \wedge T \equiv p, \quad p \vee F \equiv p.$$

بنابراین خط سوم خروجی، یعنی فرم ساده‌شده معادل، همان p است. خروجی کامل برنامه چنین خواهد بود:

$((p | F) \& (T | F))$
 $((p \& T) | F)$
 p

پیچیدگی زمانی و حافظه

اگر اندازه syntax tree ورودی را با n نشان دهیم، تجزیه فرمول، حذف عملگرهای شرطی و دوشروطی و تبدیل به NNF هر کدام در حالت معمول دارای زمان و حافظه $O(n)$ هستند.

با این حال، تولید CNF یا DNF ممکن است به علت اجرای مکرر توزیع، اندازه فرمول را به صورت نمایی افزایش دهد. بنابراین پیچیدگی زمانی و حافظه این دو مرحله در بدترین حالت نمایی است:

$$O(2^n).$$

هر بار اجرای `simplify_once` یک پیمایش روی syntax tree انجام می‌دهد. چون ساده‌سازی تا رسیدن به نقطه ثابت تکرار می‌شود، هزینه این قسمت در بدترین حالت می‌تواند بیشتر از خطی و تقریباً $O(n^2)$ باشد؛ هرچند برای فرمول‌های کوچک معمولاً بسیار سریع اجرا می‌شود.

محدودیت‌های برنامه

- نام هر متغیر فقط یک حرف کوچک انگلیسی از a تا z است.
- هر زیرفرمول دوتایی باید به‌طور کامل داخل پرانتز قرار گیرد.
- وجود فاصله درون فرمول باعث خطای تجزیه می‌شود.
- بخش ساده‌ساز فقط قوانین تعریف‌شده در کد را اعمال می‌کند و یک logical minimum form کامل تولید نمی‌کند.
- تولید CNF و DNF برای فرمول‌های بزرگ ممکن است خروجی بسیار بزرگی ایجاد کند.
- افزایش حد بازگشت به 10000 امکان پردازش فرمول‌های عمیق‌تر را فراهم می‌کند، اما ورودی بسیار عمیق همچنان می‌تواند منابع زیادی مصرف کند.